

GENERATIVE AI · 3 DAY COURSE

GenAI 3-Day 통합 학습노트

VAE → DDPM → Score-Based Model (SDE / ODE)

DAY 1 Variational Autoencoder

DAY 2 Denoising Diffusion Probabilistic Model

DAY 3 Score-Based Model · SDE · Probability Flow ODE

이론 노트북 3종 · 실습 노트북 3종 · 분포 유도 PDF 통합 정리

원 강의자료 제작: 박광석 (모두의연구소)

목차

GenAI 3-Day 통합 학습노트

VAE → DDPM → Score-Based Model (SDE / ODE)

1. 큰 그림 — 하나의 문제, 세 가지 우회로

- 1.1 공통의 목표
- 1.2 세 가지 우회 전략
- 1.3 핵심 통찰 — 셋은 결국 한 몸

2. DAY 1 — Variational Autoencoder (VAE)

- 2.1 한 줄 요약
- 2.2 세 구성요소
- 2.3 ELBO — 왜 하한을 쓰는가
- 2.4 Reparameterization Trick
- 2.5 손실 함수 (실습 코드 기준)
- 2.6 실습 코드 ↔ 수식 대응표
- 2.7 Day 1 체크 문항
- 2.8 VAE의 의의

3. DAY 2 — Denoising Diffusion Probabilistic Model (DDPM)

- 3.1 한 줄 요약
- 3.2 순방향 확산 (Forward process) — 학습 파라미터 없음
- 3.3 역방향 복원 (Reverse process) — 여기에만 θ 가 있다
- 3.4 훈련 목표 — VLB에서 L_{simple} 까지
- 3.5 ★ 사후분포 $q(x_{t-1} | x_t, x_0)$ 유도
- 3.6 U-Net의 출력 타겟 — $\epsilon / x_0 / v$
- 3.7 DDPM의 한계와 후속 모델
- 3.8 실습 코드 ↔ 수식 대응표

4. DAY 3 — Score-Based Model → SDE / ODE

- 4.1 문제의식 — 왜 또 새로운 접근인가
- 4.2 스코어(score)란
- 4.3 목표 설정 — Fisher Divergence
- 4.4 학습 방법 — Denoising Score Matching (DSM)
- 4.5 추론 — Langevin Dynamics
- 4.6 한계 → Annealed → 그래도 남은 문제
- 4.7 연속화를 위한 두 개의 열쇠
- 4.8 SDE — 노이즈 주입을 연속 시간으로
- 4.9 SDE의 두 갈래 — VP vs VE
- 4.10 Reverse SDE — 시간을 거꾸로
- 4.11 Probability Flow ODE — 무작위성 제거
- 4.12 PF-ODE가 여는 세 가지 문
- 4.13 샘플러 3종 비교
- 4.14 실습 코드 ↔ 수식 대응표 (JAX / Flax)

5. 3일 통합 — 비교표와 계보

5.1 세 모델 한눈에

5.2 계보도

5.3 세 개의 다리 (기억할 등식)

6. 직접 확인해 보기 (실행 가능)

6.1 노이즈 스케줄 — β_t, α^{-t} , SNR

6.2 순방향 확산을 2D 토이 데이터로 보기

6.3 ★ 사후분포 계수 수치 검증 (§ 3.5)

6.4 스코어 필드 — "데이터가 많은 방향"

6.5 Langevin Dynamics — 노이즈항의 역할

6.6 ★ 왜 Annealed Langevin이 필요한가 (§ 4.6의 실증)

6.7 VP-SDE vs VE-SDE — 분산 거동 비교

7. 자주 헷갈리는 지점 Q&A

8. 원본 자료 정오표 / 읽을 때 주의할 점

9. 용어 사전

10. 다음 단계 — Flow Matching

11. 참고 자료

GenAI 3-Day 통합 학습노트

VAE → DDPM → Score-Based Model (SDE / ODE)

정리 대상 원본

구분	파일	성격
DAY 1	GENAI_DAY1_VAE.ipynb / GENAI_DAY1_VAE_프로젝트_.ipynb	이론 / PyTorch 실습
DAY 2	GENAI_DAY2_DDPM.ipynb / GenAI_DAY2_DDPM_프로젝트_.ipynb	이론 / PyTorch 실습
DAY 3	GenAI_DAY3_SBM.ipynb / GenAI_DAY3_SBM_프로젝트_.ipynb	이론 / JAX·Flax 실습
보충	diffusion_분포_유도.pdf	DDPM 사후분포 $q(x_{t-1} x_t, x_0)$ 유도

원 강의자료 제작: 박광석 (모두의연구소)

이 노트의 사용법

- § 1 3일치를 하나의 문제의식으로 꿰는 지도 — 먼저 읽으세요.
- § 2~§ 4 각 Day의 수식·개념 정리 + 실습 코드 대응표.
- § 5 3모델 통합 비교 + 계보도.
- § 6 실행 가능한 시각화 셀 (numpy/matplotlib만 필요, 데이터 다운로드 없음).
- § 7 자주 헛갈리는 지점 Q&A, § 8 원본 자료 정오표, § 9 용어 사전.

§ 6 이외의 셀은 전부 설명용입니다. GPU 없이도 노트북 전체를 읽고 § 6만 실행하면 됩니다.

1. 큰 그림 — 하나의 문제, 세 가지 우회로

1.1 공통의 목표

모든 생성모델의 목표는 동일합니다.

관측 데이터가 따르는 알 수 없는 분포 $p_{\text{data}}(\mathbf{x})$ 를 $p_{\theta}(\mathbf{x})$ 로 근사하고, 거기서 샘플링한다.

정공법은 로그우도 $\log p_{\theta}(\mathbf{x})$ 를 직접 최대화하는 것입니다. 그런데 이것이 **intractable** 합니다.

Intractable?

확률을 정의하려면 전 공간에 대한 정규화(적분/전체 합)가 필요한데, 고차원에서는 이 계산량이 감당 불가능합니다.

예: $p_{\theta}(\mathbf{x}) = \int p_{\theta}(\mathbf{x} | \mathbf{z})p(\mathbf{z}) d\mathbf{z}$ — 모든 $\mathbf{z}$ 를 훑어야 합니다.

1.2 세 가지 우회 전략

	우회 방법	실제로 학습하는 것	손실 형태
VAE	계산 가능한 하한(ELBO)을 대신 최대화	인코더 $q_\phi(\mathbf{z} \mathbf{x})$ + 디코더 $p_\theta(\mathbf{x} \mathbf{z})$	재구성 + KL
DDPM	파괴 과정을 가우시안 마르코프 체인으로 고정 → 변분하한을 단순화	노이즈 예측기 $\epsilon_\theta(\mathbf{x}_t, t)$	MSE
SBM	밀도 대신 밀도의 기울기(score)만 학습 (정규화 상수 소거)	스코어망 $s_\theta(\mathbf{x}, t)$	가중 DSM

1.3 핵심 통찰 — 셋은 결국 한 몸

$$\underbrace{\nabla_{\mathbf{x}_t} \log p_{0t}(\mathbf{x}_t | \mathbf{x}_0)}_{\text{score}} = -\frac{\mathbf{x}_t - \boldsymbol{\mu}_t}{\sigma_t^2} = -\frac{\boldsymbol{\epsilon}}{\sigma_t}$$

가우시안 퍼터베이션 커널 위에서 "노이즈를 맞추는 일"(DDPM)과 "스코어를 맞추는 일"(SBM)은 상수배 차이입니다.

DDPM은 이산 시간 VP-SDE의 한 인스턴스이고, VAE는 여기에 "압축된 공간(latent)"이라는 개념을 제공해 LDM/Stable Diffusion으로 재합류합니다.

VAE (잠재공간) + DDPM (점진적 복원) + SBM (연속시간·스코어) → LDM / SD3 / Flow Matching

2. DAY 1 — Variational Autoencoder (VAE)

2.1 한 줄 요약

데이터를 **확률분포**로 인코딩하고, 그 분포에서 뽑은 잠재변수로 다시 복원한다. 잠재공간이 $\mathcal{N}(\mathbf{0}, \mathbf{I})$ 를 닮도록 규제해 두면, 학습 후에는 **노이즈만 뽑아 넣어도 새 데이터가 나온다**.

2.2 세 구성요소

요소	기호	역할	설계 이유
Prior	$p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$	잠재공간의 사전 가정	학습 후 샘플링이 쉬움
Encoder	$q_\phi(\mathbf{z} \mathbf{x})$	$\boldsymbol{\mu}_\phi, \log \sigma_\phi^2$ 출력	참 posterior $p(\mathbf{z} \mathbf{x})$ 는 intractable → 대각 가우시안 으로 근사
Decoder	$p_\theta(\mathbf{x} \mathbf{z})$	$\mathbf{z} \rightarrow \hat{\mathbf{x}}$ 복원	어떤 아키텍처든 조건부 확률 하나로 표현

왜 대각 가우시안인가: 차원 간 상관을 0으로 가정하면 KL 항이 **닫힌 형태(closed form)**로 떨어져 학습이 빠르고 수치 오차가 줄어듭니다.

왜 로그분산을 예측하는가: $\log \sigma^2$ 는 실수 전체를 자유롭게 쓸 수 있고, exp를 씌우면 분산이 항상 양수로 보장됩니다.

2.3 ELBO — 왜 하한을 쓰는가

핵심 항등식:

$$\log p_{\theta}(\mathbf{x}) = \underbrace{\text{ELBO}(q_{\phi})}_{\text{계산 가능}} + \underbrace{D_{KL}(q_{\phi}(\mathbf{z} | \mathbf{x}) \| p_{\theta}(\mathbf{z} | \mathbf{x}))}_{\geq 0, \text{ 계산 불가}}$$

KL은 항상 0 이상이므로 $\text{ELBO} \leq \log p_{\theta}(\mathbf{x})$. 즉 **ELBO를 올리면 로그우도도 따라 올라가되 절대 넘지는 못합니다.**

$$\text{ELBO}(q_{\phi}) = \underbrace{\mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x} | \mathbf{z})]}_{\text{재구성: 잘 복원하기}} - \underbrace{D_{KL}(q_{\phi}(\mathbf{z} | \mathbf{x}) \| p(\mathbf{z}))}_{\text{정규화: 잠재공간이 예쁘게}}$$

유도 (3줄 버전)

$$\log p_{\theta}(x) = \int q_{\phi}(z | x) \log p_{\theta}(x) dz = \int q_{\phi}(z | x) \log \frac{p_{\theta}(x | z)p_{\theta}(z)}{p_{\theta}(z | x)} dz$$

분자·분모에 $q_{\phi}(z | x)$ 를 곱해 정리하면

$$= \underbrace{\int q_{\phi} \log p_{\theta}(x | z) dz - D_{KL}(q_{\phi} \| p_{\theta}(z))}_{\text{ELBO}} + \underbrace{D_{KL}(q_{\phi}(z | x) \| p_{\theta}(z | x))}_{\geq 0}$$

마지막 항을 버리면 부등식이 됩니다. 이 "버리는" 행위가 VAE가 정확한 우도를 못 주는 이유이고, Day 3의 PF-ODE가 극복하는 지점입니다.

두 항의 줄다리기

- 재구성 항만 크게 하면 → 잠재공간이 흐트러져 **샘플링이 안 됨** (오토인코더로 퇴화)
- KL 항만 크게 하면 → 잠재공간이 사전분포에 갇히려 **posterior collapse**, 흐릿한 출력

2.4 Reparameterization Trick

$\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$ 를 그냥 샘플링하면 그 지점에서 **미분이 끊겨** 역전파가 불가능합니다. 무작위성을 모델 바깥으로 분리합니다.

$$\mathbf{z} = \boldsymbol{\mu}_{\phi}(\mathbf{x}) + \boldsymbol{\sigma}_{\phi}(\mathbf{x}) \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

이제 $\boldsymbol{\epsilon}$ 만 랜덤이고 $\boldsymbol{\mu}, \boldsymbol{\sigma}$ 로 가는 경로는 전부 미분 가능합니다.

이 아이디어는 Day 2에서 그대로 재등장합니다. DDPM의 $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}$ 이 정확히 같은 형태의 재파라미터화입니다.

2.5 손실 함수 (실습 코드 기준)

$$\mathcal{L} = \underbrace{\text{BCE}(\hat{\mathbf{x}}, \mathbf{x})}_{\text{reduction='sum'}} + \underbrace{\left(-\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2) \right)}_{D_{KL}(\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2) \| \mathcal{N}(\mathbf{0}, \mathbf{1})) \text{ 닫힌 형태}}$$

KL 닫힌 형태 확인: 1차원에서

$$D_{KL}(\mathcal{N}(\mu, \sigma^2) \| \mathcal{N}(0, 1)) = \frac{1}{2}(\mu^2 + \sigma^2 - 1 - \log \sigma^2).$$

부호를 정리하면 코드의 `-0.5 * sum(1 + log_var - mean^2 - exp(log_var))` 와 일치합니다.

BCE를 쓰는 이유: MNIST 픽셀을 [0, 1] 확률로 보고 베르누이 우도를 가정하기 때문입니다. 디코더 마지막이 `sigmoid` 인 것과 짝을 이룹니다.

2.6 실습 코드 ↔ 수식 대응표

(GENAI_DAY1_VAE_프로젝트_.ipynb)

코드	수식	메모
<code>Encoder.forward → mean, log_var</code>	$\mu_{\phi}(\mathbf{x}), \log \sigma_{\phi}^2(\mathbf{x})$	FC 2층 + LeakyReLU(0.2)
<code>Model.reparameterization(mean, var)</code>	$\mathbf{z} = \mu + \sigma \odot \epsilon$	두 번째 인자로 들어오는 값은 표준편차 (§ 8 정오표 참조)
<code>torch.exp(0.5 * log_var)</code>	$\sigma = \exp(\frac{1}{2} \log \sigma^2)$	로그분산 → 표준편차
<code>Decoder.forward → sigmoid</code>	$p_{\theta}(\mathbf{x} \mathbf{z})$ 의 베르누이 평균	BCE와 짝
<code>binary_cross_entropy(..., reduction='sum')</code>	재구성 항	배치 내 전 픽셀 합
<code>-0.5*torch.sum(1+log_var-mean.pow(2) - log_var.exp())</code>	KL 항	위 닫힌 형태
<code>decoder(torch.randn(B, latent_dim))</code>	$\mathbf{z} \sim p(\mathbf{z})$ 에서 생성	prior가 $\mathcal{N}(0, I)$ 이기에 가능

하이퍼파라미터: `x_dim=784` , `hidden_dim=400` , `latent_dim=200` , `batch=128` , `lr=1e-3` , `epochs=30`

`latent_dim` 은 정보 압축과 품질의 트레이드오프입니다. 너무 작으면 정보 손실로 흐려지고, 너무 크면 KL 규제가 느슨해져 잠재공간 구조가 무너집니다.

2.7 Day 1 체크 문항

Q. KL Divergence는 어떤 역할인가?

두 분포의 차이를 재는 함수로, "근사 분포가 이상적 분포를 얼마나 잘 따라가는가"를 수치화합니다.

$$D_{KL}(P\|Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} = \underbrace{\left(- \sum_x P(x) \log Q(x) \right)}_{\text{Cross-Entropy } H(P,Q)} - \underbrace{\left(- \sum_x P(x) \log P(x) \right)}_{\text{Entropy } H(P)}$$

성질: $D_{KL} \geq 0$, 등호는 $P = Q$ 일 때만. **비대칭**($D_{KL}(P\|Q) \neq D_{KL}(Q\|P)$)이라 거리(metric)는 아닙니다.

Q. VAE 샘플이 GAN보다 흐린 이유는?

① 재구성 손실(BCE/MSE)이 **평균적으로 맞히는 답**을 선호해 다봉 분포의 평균을 출력하고, ② KL 규제가 잠재공간을 뭉개며, ③ ELBO가 하한이라 우도 최적점에 정확히 도달하지 않기 때문입니다.

2.8 VAE의 의의

- 확률적 인코딩·디코딩을 **하나의 신경망 안에서 함께 학습**한 최초 구조
- 재파라미터화로 **저분산·안정적 그라디언트** 확보 → 대규모 확장 가능
- KL 정규화로 잠재공간이 **연속적·구조화된 의미 공간**이 됨 → 보간, 의미 조작
- 이 세 가지가 그대로 LDM(Stable Diffusion)의 1단계 압축기로 계승됩니다.

3. DAY 2 — Denoising Diffusion Probabilistic Model (DDPM)

3.1 한 줄 요약

데이터를 **아주 조금씩 T번 노이즈로 망가뜨린 뒤**, 그 과정을 한 칸씩 되돌리는 법을 신경망에 가르친다. 생성은 순수 노이즈에서 시작해 T번 되감기.

VAE가 "공간을 압축"했다면 DDPM은 "**시간을 꼬집는다.**" 한 번에 어려운 문제(노이즈→이미지)를 T개의 쉬운 문제(조금 덜 노이즈로)로 분해하는 것이 본질입니다.

3.2 순방향 확산 (Forward process) — 학습 파라미터 없음

마르코프 체인으로 정의합니다.

$$q(\mathbf{x}_1, \dots, \mathbf{x}_T | \mathbf{x}_0) := \prod_{t=1}^T q(\mathbf{x}_t | \mathbf{x}_{t-1}), \quad q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

한 스텝의 의미는 두 줄입니다.

1. **스케일 축소**: $\sqrt{1 - \beta_t}$ 를 곱해 원본 기여도를 살짝 줄이고
2. **노이즈 주입**: 분산 β_t 의 가우시안을 더한다

β_t 를 아주 작게 잡고 T 번 누적하면 $\mathbf{x}_T \rightarrow \mathcal{N}(\mathbf{0}, \mathbf{I})$ 로 수렴합니다.

닫힌 형태 — 중간 단계를 건너뛰기

$\alpha_t := 1 - \beta_t$, $\bar{\alpha}_t := \prod_{s=1}^t \alpha_s$ 로 두면

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}) \iff \mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

이 한 줄이 DDPM 학습을 가능하게 합니다. t 를 1~T 중 아무거나 뽑아서 곧바로 $\mathbf{x}_t$ 를 만들 수 있으므로, 매 배치마다 T번 시뮬레이션할 필요가 없습니다.

기호	의미
β_t	스텝 t 에서 얼마나 노이즈를 넣을지 (스케줄, 보통 $10^{-4} \rightarrow 2 \times 10^{-2}$ 선형)
$\alpha_t = 1 - \beta_t$	한 스텝 후 살아남는 신호 비율
$\bar{\alpha}_t$	t 시점까지 누적 생존율 — $\bar{\alpha}_t \rightarrow 0$ 이면 완전 노이즈
ϵ	주입된 노이즈 자체 — 모델이 맞춰야 할 정답 라벨

3.3 역방향 복원 (Reverse process) — 여기에만 θ 가 있다

참 역방향 분포 $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$ 는 데이터 분포 전체를 알아야 해서 계산 불가. 신경망으로 근사합니다.

$$p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\mu_{\theta}(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I})$$

- σ_t^2 는 이론적 닫힌 형태가 있어 **고정 상수**로 두는 경우가 많습니다.
- 모델이 실제로 예측하는 것은 평균 μ_{θ} 인데, **노이즈 ϵ 예측으로 재파라미터화**하면 목표가 단순 MSE가 되어 학습이 안정됩니다.

기호 θ 가 보이면 "학습 가능한 파라미터가 있다"는 신호입니다. 순방향 q 에는 θ 가 없습니다.

3.4 훈련 목표 — VLB에서 L_{simple} 까지

VAE의 ELBO에서 영감을 얻어 변분하한을 세우면:

$$\mathbb{E}_q \left[\underbrace{D_{KL}(q(\mathbf{x}_T | \mathbf{x}_0) \| p(\mathbf{x}_T))}_{L_T: \text{상수, 학습 불필요}} + \underbrace{\sum_{t>1} D_{KL}(q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \| p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t))}_{L_{t-1}: \text{핵심}} - \underbrace{\log p_\theta(\mathbf{x}_0 | \mathbf{x}_1)}_{L_0: \text{재구성}} \right]$$

VAE의 (재구성 + 정규화) 구조가 **T개로 늘어난 형태**입니다.

L_{t-1} 은 **두 가우시안 사이의 KL**이므로 닫힌 형태로 평균의 차이만 남고, 가중치를 떼어내면 최종적으로:

$$L_{\text{simple}} = \mathbb{E}_{t, \mathbf{x}_0, \epsilon} \left[\left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|_2^2 \right]$$

"내가 넣은 노이즈를 맞혀라" 는 단순 MSE. t 는 매 미니배치마다 $\{1, \dots, T\}$ 에서 균등 샘플링하므로, 한 번의 역전파로 여러 시간대를 섞어 학습합니다.

여기서 L_{t-1} 을 계산하려면 $q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)$ 를 **명시적으로 알아야 합니다**. 그것이 다음 절 (§ 3.5)의 유도입니다.

3.5 ★ 사후분포 $q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)$ 유도

(diffusion_분포_유도.pdf 정리 + 계수 계산)

(1) 전제 — 왜 계산이 가능한가

1. 각 스텝이 **선형 가우시안**: $q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$ — 스케일링 + 가우시안 노이즈
2. 가우시안의 성질
 - 가우시안 변수에 상수를 곱해도 가우시안
 - 가우시안끼리 더해도 가우시안
 - 여러 가우시안의 조건부 분포도 가우시안
3. 따라서 $\mathbf{x}_0 \rightarrow \mathbf{x}_1 \rightarrow \dots \rightarrow \mathbf{x}_T$ 전체가 **결합 가우시안(jointly Gaussian)**. 이 중 어떤 조합의 조건부를 만들어도 결과는 항상 가우시안이고, **근사 없이 정확히** 계산됩니다.

우리가 이미 아는 세 개의 가우시안:

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}), \quad q(\mathbf{x}_{t-1} | \mathbf{x}_0), \quad q(\mathbf{x}_t | \mathbf{x}_0)$$

닫힌 형태(closed form)란? 적분이나 반복 계산 없이 대입만 하면 답이 나오는 명시적 수식. 2차방정식의 근의 공식 같은 것.

(2) 베이즈 정리

$$q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{x}_0) q(\mathbf{x}_{t-1} | \mathbf{x}_0)}{q(\mathbf{x}_t | \mathbf{x}_0)}$$

마르코프 성질에 의해 $q(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{x}_0) = q(\mathbf{x}_t | \mathbf{x}_{t-1})$.

($\mathbf{x}_t$ 는 $\mathbf{x}_{t-1}$ 만 알면 결정되고, $\mathbf{x}_0$ 의 정보는 이미 $\mathbf{x}_{t-1}$ 에 녹아 있으므로 추가 정보가 아님)

$\mathbf{x}_{t-1}$ 에 대한 함수로 볼 때 분모는 상수이므로:

$$q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \propto q(\mathbf{x}_t | \mathbf{x}_{t-1}) q(\mathbf{x}_{t-1} | \mathbf{x}_0)$$

(3) 지수부만 떼어 전개

가우시안 밀도는 $\exp(-\frac{1}{2}(\cdot)^2/\sigma^2)$ 꼴이므로, 세 분포의 지수부를 모으면

$$q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \propto \exp \left[-\frac{1}{2} \left\{ \frac{(\mathbf{x}_t - \sqrt{\alpha_t} \mathbf{x}_{t-1})^2}{\beta_t} + \frac{(\mathbf{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0)^2}{1 - \bar{\alpha}_{t-1}} - \frac{(\mathbf{x}_t - \sqrt{\bar{\alpha}_t} \mathbf{x}_0)^2}{1 - \bar{\alpha}_t} \right\} \right]$$

세 번째 항에는 $\mathbf{x}_{t-1}$ 이 없으므로 **상수 취급**합니다.

(4) $\mathbf{x}_{t-1}$ 에 대한 2차식으로 정리

$$\propto \exp \left(-\frac{1}{2} \left[\underbrace{\left(\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}} \right)}_A \mathbf{x}_{t-1}^2 - 2 \underbrace{\left(\frac{\sqrt{\alpha_t}}{\beta_t} \mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}} \mathbf{x}_0 \right)}_B \mathbf{x}_{t-1} + C \right] \right)$$

$-\frac{1}{2} [A \mathbf{x}_{t-1}^2 - 2B \mathbf{x}_{t-1} + C]$ 를 가우시안 표준형 $-\frac{(\mathbf{x}_{t-1} - \mu)^2}{2\sigma^2}$ 과 비교하면

$$\sigma^2 = \frac{1}{A}, \quad \mu = \frac{B}{A}$$

(5) 계수 계산

분산

$$A = \frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}} = \frac{\alpha_t(1 - \bar{\alpha}_{t-1}) + \beta_t}{\beta_t(1 - \bar{\alpha}_{t-1})}$$

분자 = $\alpha_t - \alpha_t \bar{\alpha}_{t-1} + 1 - \alpha_t = 1 - \bar{\alpha}_t$ (since $\alpha_t \bar{\alpha}_{t-1} = \bar{\alpha}_t$). 따라서

$$\tilde{\beta}_t = \sigma^2 = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$$

평균

$$\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0 + \frac{\sqrt{\alpha_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t$$

최종

$$q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I})$$

(6) 왜 이게 중요한가

이 결과가 **DDPM 학습의 정답지**입니다. L_{t-1} 은 이 분포와 p_θ 사이의 KL이고, 두 가우시안의 분산을 고정하면 KL은 **평균 차이의 제곱**으로 축소됩니다. 여기에 $\mathbf{x}_0 = \frac{\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \epsilon}{\sqrt{\bar{\alpha}_t}}$ 를 대입하면 $\tilde{\mu}_t$ 가 ϵ 만의 식으로 정리되고, 결국 $\|\epsilon - \epsilon_\theta\|^2$ 가 나옵니다. § 3.4의 L_{simple} 이 하늘에서 떨어진 것이 아니라 여기서 온 것입니다.

3.6 U-Net의 출력 타깃 — $\epsilon / \mathbf{x}_0 / \mathbf{v}$

순방향 상태를 $\mathbf{x}_t = \alpha \mathbf{x}_0 + \sigma \epsilon$ ($\alpha = \sqrt{\bar{\alpha}_t}$, $\sigma = \sqrt{1 - \bar{\alpha}_t}$) 로 두면, 세 선택지는 **서로 선형변환으로 오갈 수 있어 수학적으로 동등**합니다.

타깃	모델 출력	변환식	특징
(A) ϵ -prediction	$\epsilon_\theta(\mathbf{x}_t, t)$	$\hat{\mathbf{x}}_0 = \frac{\mathbf{x}_t - \sigma \epsilon_\theta}{\alpha}$	가장 표준적, 구현 단순. 단 t 가 작을 때 불안정
(B) $\mathbf{x}_0$ -prediction	$\hat{\mathbf{x}}_0(\mathbf{x}_t, t)$	$\hat{\epsilon} = \frac{\mathbf{x}_t - \alpha \hat{\mathbf{x}}_0}{\sigma}$	t 가 클 때 불안정
(C) $\mathbf{v}$ -prediction	$\mathbf{v} = \alpha \epsilon - \sigma \mathbf{x}_0$	$\hat{\mathbf{x}}_0 = \alpha \mathbf{x}_t - \sigma \mathbf{v}, \quad \hat{\epsilon} = \sigma \mathbf{x}_t + \alpha \mathbf{v}$	전 구간 균형, 고해상도·distillation에서 유리

실무 순서: ϵ -prediction으로 시작 → 안정성 튜닝 단계에서 $\mathbf{v}$ -prediction 시도.

샘플링의 핵심 — 역이동 평균식

어떤 타깃을 쓰든 $\hat{\epsilon}$ 만 얻으면 다음 식으로 $\mathbf{x}_{t-1}$ 을 계산합니다.

$$\mu_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \hat{\epsilon} \right)$$

$$\mathbf{x}_{t-1} = \mu_\theta(\mathbf{x}_t, t) + \sigma_t \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

3.7 DDPM의 한계와 후속 모델

한계 두 가지

1. 느린 속도 — 이미지 한 장에 1,000회 이상 순차 반복 (마르코프 체인이라 건너뛸 수 없음)
2. 큰 계산량 — 픽셀 공간에서 전부 계산, 해상도가 오르면 폭증

DDIM — "학습은 그대로, 샘플링만 바꾼다"

핵심은 **비-마르코프(Non-Markovian) 추론 경로**입니다. $\mathbf{x}_{t-1}$ 을 구할 때 $\mathbf{x}_t$ 뿐 아니라 예측된 $\mathbf{x}_0$ 를 동시에 고려합니다.

$$\mathbf{x}_{t-1} = \underbrace{\sqrt{\bar{\alpha}_{t-1}} \left(\frac{\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(\mathbf{x}_t, t)}{\sqrt{\bar{\alpha}_t}} \right)}_{\text{predicted } \mathbf{x}_0} + \underbrace{\sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \epsilon_\theta(\mathbf{x}_t, t)}_{\text{direction pointing to } \mathbf{x}_t} + \sigma_t \mathbf{z}$$

σ_t 는 **조절 가능한 하이퍼파라미터**입니다.

- $\sigma_t = \sqrt{\frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}} \sqrt{1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}} \rightarrow$ **DDPM과 동일**

- $\sigma_t = 0 \rightarrow$ 무작위항 소멸, **결정론적(deterministic)** 샘플링

핵심 이점:

- 손실함수가 **DDPM의 L_{simple} 과 동일** → 학습된 DDPM 체크포인트를 그대로 쓰고 샘플링 수식만 교체하면 즉시 10~100배 가속

- **역전성(Inversion):** 경로가 결정론적이라 이미지→노이즈를 정확히 되돌릴 수 있음 → 원본 특징을 유지한 편집/스타일 변환

§ 4의 Probability Flow ODE 관점에서 보면, DDIM은 사실상 PF-ODE를 1차 이산화한 솔버입니다.

LDM — "픽셀 대신 잠재공간에서 확산"

디지털 데이터의 대부분은 인간이 지각하기 어려운 미세 디테일에 할당됩니다. 이를 두 단계로 분리합니다.

단계	담당	하는 일
지각적 압축 (Perceptual)	VAE Encoder $\mathcal{E}$	시각적으로 중요치 않은 고주파 정보 제거
의미적 압축 (Semantic)	Diffusion U-Net	압축된 z 공간에서 의미 구조 생성

512 × 512 를 64 × 64 잠재공간으로 옮기면 데이터 크기가 **64배** 줄어듭니다.

구조 3분할:

- **Pixel Space**: $x \xrightarrow{\mathcal{E}} z, z \xrightarrow{\mathcal{D}} \tilde{x}$
- **Latent Space**: 순방향 확산 + Denoising U-Net (내부에 **Cross-Attention** Q, K, V)
- **Conditioning**: 텍스트·이미지·시맨틱맵 y 를 도메인 인코더 τ_θ 로 처리해 U-Net에 주입

여기서 **Day 1**의 VAE가 다시 등장한다는 점이 중요합니다. Stable Diffusion = VAE(압축) + DDPM(생성) + Cross-Attention(조건화).

3.8 실습 코드 ↔ 수식 대응표

(GenAI_DAY2_DDPM_프로젝트_.ipynb)

코드	수식 / 개념
<code>SinusoidalPosEmb</code>	타임스텝 t 를 sin/cos 주기 패턴으로 임베딩 (Transformer 위치 인코딩 차용)
<code>ConvBlock(..., residual=True)</code> 에서 <code>y + time_emb</code>	U-Net에 시간 정보 주입
<code>betas = torch.linspace(1e-4, 2e-2, n_times)</code>	선형 β 스케줄
<code>self.alphas = 1 - betas</code>	$\alpha_t = 1 - \beta_t$
<code>alpha_bars = torch.cumprod(self.alphas, dim=0)</code>	$\bar{\alpha}_t = \prod_{s \leq t} \alpha_s$
<code>make_noisy(x_zeros, t)</code>	$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$
<code>extract(a, t, x_shape)</code>	배치별 서로 다른 t 의 계수를 broadcasting 형태로 뽑기
<code>scale_to_minus_one_to_one</code>	입력을 $[-1, 1]$ 로 정규화 (가우시안 가정과 정합)
<code>denoising_loss(pred_epsilon, epsilon)</code>	L_{simple}
<code>diffusion.sample(N)</code>	$t = T \rightarrow 1$ 역방향 반복

축소 하이퍼파라미터 (수업용): `img_size=(14,14,1)` , `n_timesteps=300` , `n_layers=4` , `hidden_dim=128` , `epochs=50` , `lr=1e-4`

원 논문 설정 (주석 처리됨): `(28,28,1)` , `n_timesteps=1000` , `n_layers=8` , `hidden_dim=256` , `epochs=200` , `lr=5e-5`

축소판은 결과가 흐릿합니다. **모델이 틀린 게 아니라 T 와 해상도를 줄인 대가**입니다. GPU 여유가 있으면 주석 블록으로 되돌려 비교해 보세요.

4. DAY 3 — Score-Based Model → SDE / ODE

4.1 문제의식 — 왜 또 새로운 접근인가

VAE는 정확한 우도를 위해 구조를 심하게 제약하거나 ELBO라는 우회로를 써야 했고, GAN은 두 모델의 경쟁이 줄타기처럼 불안정했습니다. 여기서 근본적인 질문이 나옵니다.

"확률 분포 $p(\mathbf{x})$ 전체를 알 필요가 있을까? 지금 서 있는 지점에서 '데이터가 더 많은 방향'만 알면 되지 않을까?"

밀도 자체가 아니라 **로그 밀도의 기울기**를 모델링하는 Score Matching이 여기서 태어납니다.

4.2 스코어(score)란

$$\text{score}(\mathbf{x}) = \nabla_{\mathbf{x}} \log p(\mathbf{x})$$

안개 낀 산에서 지도는 없지만 발밑 경사를 느껴 높은 쪽으로 한 발씩 옮기면 정상에 도착합니다. 그 "**더 높은 곳을 가리키는 화살표**"가 스코어입니다.

정규분포에서 확인해 보면: 좌우 끝에서는 화살표가 중앙을 향하고, 정점에서는 길이가 0입니다.

결정적 이점 — 정규화 상수가 사라진다

$p(\mathbf{x}) = \frac{\tilde{p}(\mathbf{x})}{Z}$ 라 할 때

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}) = \nabla_{\mathbf{x}} \log \tilde{p}(\mathbf{x}) - \underbrace{\nabla_{\mathbf{x}} \log Z}_{=0}$$

intractable했던 Z 가 미분 과정에서 그냥 없어집니다. 이것이 스코어를 쓰는 근본 이유입니다.

NCSN (Noise Conditional Score Network)은 이 화살표를 즉석에서 계산해 주는 신경망입니다.

- 입력: 현재 위치 $\mathbf{x}$, 노이즈 세기 σ (또는 시간 t)
- 출력: 그 지점의 스코어 벡터

4.3 목표 설정 — Fisher Divergence

$$J(\theta) = \mathbb{E}_{p(\mathbf{x})} \left[\frac{1}{2} \|\nabla_{\mathbf{x}} \log p(\mathbf{x}) - s_{\theta}(\mathbf{x})\|_2^2 \right]$$

MSE와 같은 꼴입니다. 다만 정답 $\nabla \log p(\mathbf{x})$ 를 우리가 모른다는 문제가 남습니다.

4.4 학습 방법 — Denoising Score Matching (DSM)

길을 일부러 잃게 만든 뒤, 집으로 돌아오는 법을 가르친다.

1. **Noise Injection** — 데이터 $\mathbf{x}_0$ 에 세기 σ 의 노이즈를 섞어 $\hat{\mathbf{x}}$ 를 만듭니다. 데이터가 없는 저밀도 영역에서도 스코어를 정의할 수 있게 하는 '훈련용 안개'.
2. **정답 확보** — 우리가 **직접 섞은** 노이즈이므로 정답 스코어를 안다:

$$\nabla_{\tilde{\mathbf{x}}} \log p_{\sigma}(\tilde{\mathbf{x}} | \mathbf{x}_0) = -\frac{\tilde{\mathbf{x}} - \mathbf{x}_0}{\sigma^2} = -\frac{\mathbf{z}}{\sigma}, \quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

3. **Optimization** — 모델의 화살표가 이 복원 방향과 일치하도록 θ 를 업데이트.

$$\mathcal{L}_{\text{DSM}} = \mathbb{E}_{t, \mathbf{x}_0, \mathbf{z}} \left[\lambda(t) \left\| s_{\theta}(\tilde{\mathbf{x}}, t) + \frac{\mathbf{z}}{\sigma_t} \right\|^2 \right]$$

★ **Day 2와의 연결**: DDPM이 노이즈 ϵ 을 예측한 행위는, 알고 보면 **가장 계산하기 쉬운 노이즈를 빌려 가장 계산하기 어려운 스코어를 간접 학습**하고 있었던 것입니다.

$$s_{\theta}(\mathbf{x}_t, t) \approx -\frac{\epsilon_{\theta}(\mathbf{x}_t, t)}{\sqrt{1 - \bar{\alpha}_t}}$$

4.5 추론 — Langevin Dynamics

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \epsilon \nabla_{\mathbf{x}} \log p_{\theta}(\mathbf{x}_i) + \sqrt{2\epsilon} \mathbf{z}_i, \quad \mathbf{z}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

항	역할
$\mathbf{x}_i$	현재 상태 (처음엔 무작위 노이즈)
$\epsilon \nabla_{\mathbf{x}} \log p_{\theta}$	밀도가 높은 방향으로 이동 (스코어를 따라감)
$\sqrt{2\epsilon} \mathbf{z}_i$	무작위 탐색 — 다양성 확보 + 지역 최적점 탈출

노이즈항이 왜 필요한가: 스코어만 따라가면 샘플들이 가장 가까운 봉우리에 **전부 뭉쳐버립니다**(mode collapse처럼). 노이즈를 섞어야 확률 지형을 충분히 탐색하며 실제 밀도를 재현합니다.

4.6 한계 → Annealed → 그래도 남은 문제

한계: 실제 데이터는 매우 좁은 매니폴드에 밀집해 있어, 그 바깥 저밀도 영역에서는 스코어가 제대로 학습되지 않습니다. 초기 노이즈 지점이 데이터 산맥에서 너무 멀면 나침반이 엉뚱한 곳을 가리킵니다.

Annealed Langevin Dynamics: 여러 세기의 σ 를 조건으로 학습. 처음엔 짙은 안개(강한 노이즈)로 대략적 형태를 잡고, 점차 안개를 걷어내며(약한 노이즈) 디테일을 찾습니다.

남은 숙제: "노이즈 레벨을 몇 단계로 나눌 것인가?" — 레벨이 **이산적**이라 그 사이의 변화를 포착하지 못합니다. 계단이 끊어져 있는 셈이죠.

"무한히 연속적으로 만들면 어떨까?" → 이 질문이 SDE/ODE 기반 생성모델의 출발점입니다.

4.7 연속화를 위한 두 개의 열쇠

열쇠 1 — Neural ODE

ResNet의 잔차 연결:

$$h_{t+1} = h_t + f(h_t, \theta_t)$$

레이어를 무한히 얇게, 무한히 많이 쌓으면:

$$\frac{h(t + \Delta t) - h(t)}{\Delta t} = f(h(t), t, \theta) \xrightarrow{\Delta t \rightarrow 0} \frac{dh(t)}{dt} = f(h(t), t, \theta)$$

이제 신경망 f 는 "다음 층의 값"이 아니라 **"현재 위치에서 데이터가 흘러갈 방향"** 을 출력합니다. '층'이라는 개념이 **시간 t 에 따른 궤적(trajecory)** 으로 바뀝니다.

	이산 (ResNet)	연속 (Neural ODE)
입력	0번째 층	$t = 0$ 의 시작점
학습	각 층의 가중치	미끄럼틀의 곡률 f
출력	마지막 층	ODE Solver로 $t = T$ 까지 적분한 결과

열쇠 2 — Normalizing Flow

$$p(x) = p(z) \cdot \left| \det \left(\frac{\partial f^{-1}}{\partial x} \right) \right|$$

야코비안 행렬식은 **공간이 얼마나 늘어나거나 줄었는지의 보정 계수**입니다. 찢음을 얇게 퍼면 밀도를 낮추고, 뭉치면 높입니다. 단 f^{-1} 이 필요하므로 **가역(1:1) 구조**라는 강한 제약이 붙습니다.

여기에 Neural ODE를 얹으면 이산적 곱셈이 **연속적 적분**으로 바뀝니다 (CNF):

$$\log p(x(T)) = \log p(x(0)) - \int_0^T \text{Tr} \left(\frac{\partial f}{\partial x(t)} \right) dt$$

4.8 SDE — 노이즈 주입을 연속 시간으로

$$dx = \underbrace{f(x, t)}_{\text{drift}} dt + \underbrace{g(t)}_{\text{diffusion}} dw$$

항	의미
$f(x, t)$ 드리프트 계수	결정론적 흐름의 경향성. 스케일 조절, 원점으로 끌어당김
$g(t)$ 확산 계수	주입 노이즈의 강도
dw 위너 프로세스	브라운 운동. 평균 0, 분산 dt 인 가우시안

퍼터베이션 커널 (Perturbation Kernel)

$$p_{0t}(x_t \mid x_0)$$

" x_0 를 들고 SDE라는 노이즈 터널에 들어가면 t 초 뒤 손에 들려 있을 이미지는?"에 대한 확률적 답안지입니다.

선형 SDE의 마법: 이 커널이 **항상 가우시안**입니다. 두 가지 이점이 따라옵니다.

- 단번의 점프** — 수만 번의 미세 시뮬레이션 없이 $x_0 \rightarrow x_t$ 를 closed-form으로 직행
- 정답 스코어의 실시간 계산** —

$$\nabla_{x_t} \log p_{0t}(x_t \mid x_0) = \nabla_{x_t} \left(-\frac{\|x_t - \mu_t\|^2}{2\sigma_t^2} + C \right) = -\frac{x_t - \mu_t}{\sigma_t^2}$$

DDPM의 닫힌 형태 $q(x_t \mid x_0)$ 가 바로 이 커널의 이산 버전입니다. **§ 3.2와 § 4.8은 같은 것을 다른 언어로 말하고 있습니다.**

4.9 SDE의 두 갈래 — VP vs VE

f 와 g 를 어떻게 잡느냐에 따라 데이터가 파괴되는 방식이 완전히 달라집니다.

	VP-SDE (Variance Preserving)	VE-SDE (Variance Exploding)
기원	DDPM의 연속시간 확장	NCSN / Score Matching의 일반화
드리프트 f	$-\frac{1}{2}\beta(t)\mathbf{x}$	0
확산 g	$\sqrt{\beta(t)}$	$\sqrt{\frac{d[\sigma^2(t)]}{dt}}$
분산 거동	전체 분산이 일정하게 유지	시간에 따라 폭발적으로 증가
$t = T$ 극한	$\mathcal{N}(\mathbf{0}, \mathbf{I})$ 로 수렴	매우 큰 분산의 가우시안
장점	수치적으로 안정, DDPM 수식과 완벽 호환	저밀도·진공 영역까지 광범위하게 스코어 학습
대표	Stable Diffusion 등 고해상도 표준	Yang Song의 NCSN++

둘은 "노이즈를 어떤 속도로, 어떤 모양으로 들이부을 것인가"에 대한 전략 차이일 뿐, 목적지는 똑같이 스코어입니다.

4.10 Reverse SDE — 시간을 거꾸로

$$d\mathbf{x} = [f(\mathbf{x}, t) - g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x})] dt + g(t) d\bar{\mathbf{w}}$$

항	역할
$f(\mathbf{x}, t)$	순방향의 배경 흐름(관성)은 역방향에도 남아 있음
$-g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x})$	스코어 . 마이너스는 "파괴의 반대 = 복원 방향"으로 힘을 가한다는 뜻. $g(t)^2$ 는 노이즈가 질을수록 나침반을 더 강하게 따르라는 가중치
$g(t)d\bar{\mathbf{w}}$	교정 노이즈 . 파괴용이 아니라, 샘플이 한 점으로 뭉치지 않고 실제 분포처럼 풍성하게 퍼지게 하는 진동

여기서 우리가 모르는 것은 $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ 하나뿐이고, 그것을 $s_\theta(\mathbf{x}, t)$ 로 대체하면 생성이 완성됩니다.

4.11 Probability Flow ODE — 무작위성 제거

$$d\mathbf{x} = \left[f(\mathbf{x}, t) - \frac{1}{2}g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \right] dt$$

무작위항 $d\bar{\mathbf{w}}$ 를 제거했는데도 시간에 따른 확률 밀도 $p_t(\mathbf{x})$ 의 변화는 SDE와 완벽히 동일합니다.

왜 가능한가 — Fokker-Planck 방정식

수천 명이 광장에서 제각기 무작위로 흩어질 때, 개인의 경로는 예측 불가하지만 '전체가 퍼져 있는 모양'은 통계적으로 일정하게 변합니다. Fokker-Planck는 이 '전체 모양의 시간 변화'를 기술하는 식입니다.

핵심은, 같은 밀도 변화를 만드는 방법이 '무작위 이동'만 있는 게 아니라는 사실입니다. 연구자들은 **노이즈 없이 정해진 레일 위를 매끄럽게 달리면서도 전체 분포는 동일하게 유지하는 속도 벡터**를 찾아냈습니다.

★ 왜 계수가 1 에서 1/2 로 줄어드는가

기존 SDE에서는 무작위 노이즈가 입자를 사방으로 흩뜨리며 **분산을 키우는 역할**을 했습니다. ODE에서는 이 노이즈를 아예 제거했으므로 분산이 자연 증가하지 않습니다. 따라서 SDE와 똑같은 확률 밀도를 유지하려면 **모아주는 힘인 스코어의 세기도 정확히 절반으로 줄여야** 수지가 맞습니다.

(원 논문: *Score-Based Generative Modeling through SDEs*, Appendix D)

4.12 PF-ODE가 여는 세 가지 문

	내용	결과
1. 정확한 역전 (Exact Inversion)	$\mathbf{x}_0 \leftrightarrow \mathbf{x}_T$ 가 단 하나의 경로로 고정	원본의 구조·형태를 완벽히 유지한 정교한 편집
2. 초고속 샘플링	궤적이 결정론적·매끄러움 → 고차 ODE Solver가 큰 보폭으로 점프 가능	DPM-Solver, UniPC 등으로 10~20 스텝 에 고품질
3. 정확한 로그우도	궤적을 따라 밀도 변화를 적분	VAE의 ELBO 같은 하한이 아닌 정확한 값

§ 2.3에서 "버렸던" KL 항이 여기서 회수됩니다. VAE가 포기했던 정확한 우도를 PF-ODE는 되찾습니다. 3일 과정의 서사가 여기서 닫힙니다.

4.13 샘플러 3종 비교

([GenAI_DAY3_SBM_프로젝트_.ipynb](#))

샘플러	방식	수식	특징
Euler-Maruyama	Reverse SDE를 1차 이산화	$\mathbf{x}_{t-\Delta t} = \mathbf{x}_t + \sigma^{2t} s_\theta \Delta t + \sigma^t \sqrt{\Delta t} \mathbf{z}$	가장 단순, 스텝 수 많이 필요
Predictor-Corrector (PC)	예측 후 Langevin MCMC로 교정	corrector: $\mathbf{x} \leftarrow \mathbf{x} + \epsilon s_\theta + \sqrt{2\epsilon} \mathbf{z}$	같은 스텝 수에서 품질 ↑
Probability Flow ODE	무작위성 제거 후 ODE Solver (RK45)	$d\mathbf{x} = -\frac{1}{2}\sigma^{2t} s_\theta dt$	결정론적, 적은 평가 횟수, 우도 계산 가능

PC 샘플러 Q&A

Q. 왜 Prediction이 아니라 Correction이 먼저 수행되는가?

현재 시간 t 에서의 샘플을 **먼저 그 시간대의 분포에 맞게 정제**한 뒤 다음 시간대로 넘어가기 위해서입니다. 현재 스텝의 오차를 줄이고 이동하는 순서입니다.

Q. SNR(signal-to-noise ratio)을 바꾸면?

너무 작으면 교정 효과가 미미하고, 너무 크면 노이즈가 과해져 이미지가 뭉개집니다. 일반적으로 **0.1~0.2**를 사용합니다 (실습 기본값 0.16).

동적 스텝 사이즈는 고정값 대신 현재 그래디언트 크기([grad_norm](#))와 노이즈 크기([noise_norm](#))를 비교해 계산하므로, 노이즈 수준 σ^t 에 맞는 보폭이 자동 결정됩니다.

4.14 실습 코드 ↔ 수식 대응표 (JAX / Flax)

이 실습의 순방향 SDE는 **VE-SDE**입니다: $dx = \sigma^t dw$

코드	수식
<code>diffusion_coeff(t, sigma) = sigma**t</code>	$g(t) = \sigma^t$
<code>marginal_prob_std(t, sigma) = sqrt((sigma**(2t)-1)/(2*log(sigma)))</code>	커널 표준편차 $\sigma_t = \sqrt{\frac{\sigma^{2t} - 1}{2 \log \sigma}}$
<code>perturbed_x = x + z * std</code>	$\tilde{\mathbf{x}} = \mathbf{x}_0 + \sigma_t \mathbf{z}$
<code>loss = mean(sum((score*std + z)**2))</code>	DSM 손실. $s_\theta \approx -\mathbf{z}/\sigma_t$ 이므로 $\sigma_t s_\theta + \mathbf{z} \rightarrow 0$. 가중치 $\lambda(t) = \sigma_t^2$
<code>GaussianFourierProjection</code>	시간 t 의 랜덤 푸리에 임베딩 (Day 2의 SinusoidalPosEmb 대응)
<code>ScoreNet(...) / marginal_prob_std</code>	출력을 σ_t 로 나눠 스케일 보정 — 노이즈 세기별 크기 차이 흡수
<code>mean_x = x + (g**2) * score * step_size</code>	$\mathbf{x} + \sigma^{2t} s_\theta \Delta t$
<code>x = mean_x + sqrt(step_size)*g*normal(...)</code>	$+ \sigma^t \sqrt{\Delta t} \mathbf{z}$
<code>ode_func</code> + <code>scipy RK45</code>	PF-ODE 수치적분

JAX 사용 시 주의점 (실습 도입부)

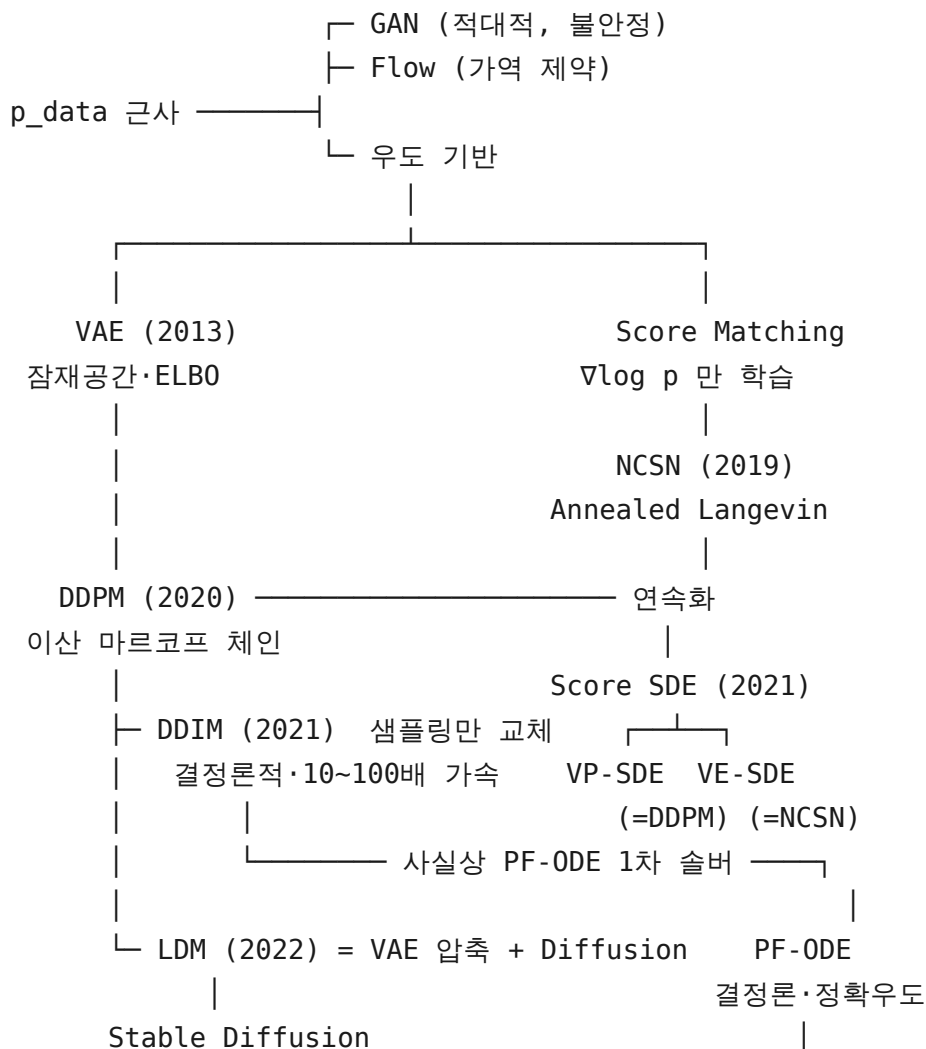
원칙	내용
함수형 (순수 함수)	전역 상태 변경 금지. 같은 입력 → 항상 같은 출력
불변성 (Immutable)	<code>x[0] = 10</code> 불가 → <code>x.at[0].set(10)</code> 으로 새 배열 반환
명시적 난수	<code>jax.random.PRNGKey(0)</code> 로 키를 만들고 <code>split</code> 으로 쪼갬. 매 스텝 새 키 필요
pmap 병렬	<code>jax.local_device_count()</code> 만큼 데이터 분할, <code>flax.jax_utils.replicate</code> 로 파라미터 복제
데이터 레이아웃	PyTorch (B,C,H,W) → JAX (B,H,W,C). <code>x.permute(0,2,3,1)</code> 필요

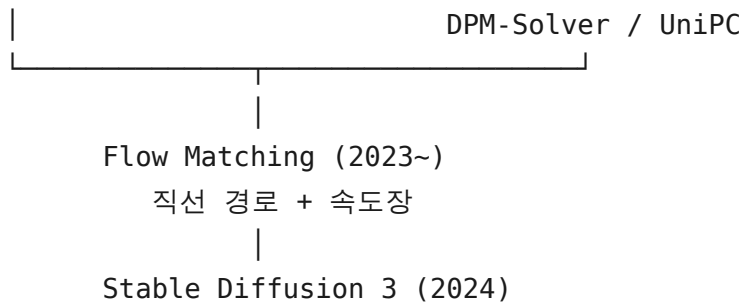
5. 3일 통합 — 비교표와 계보

5.1 세 모델 한눈에

	VAE	DDPM	SBM (SDE/ODE)
학습 대상	q_ϕ, p_θ	$\epsilon_\theta(\mathbf{x}_t, t)$	$s_\theta(\mathbf{x}, t)$
목적함수	ELBO (재구성 + KL)	L_{simple} (MSE)	가중 DSM
시간 개념	없음 (1스텝)	이산 $t = 1..T$	연속 $t \in [0, T]$
잠재변수	저차원 $\mathbf{z}$	$\mathbf{x}_t$, 원본과 동일 차원	$\mathbf{x}_t$, 동일 차원
생성 속도	매우 빠름 (1회)	느림 (T회)	솔버에 따라 조절 (10~1000)
샘플 품질	흐릿함	높음	매우 높음
우도 계산	하한(ELBO)만	하한(VLB)만	PF-ODE로 정확히
대표 약점	blurriness, posterior collapse	속도, 계산량	솔버/스케줄 설계 복잡

5.2 계보도





5.3 세 개의 다리 (기억할 등식)

- 1) $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$ — VAE의 재파라미터화 = DDPM의 순방향 커널
- 2) $s_\theta(\mathbf{x}_t, t) \approx -\frac{\epsilon_\theta(\mathbf{x}_t, t)}{\sqrt{1 - \bar{\alpha}_t}}$ — 노이즈 예측 = 스코어 예측
- 3) DDPM = VP-SDE의 이산화, NCSN = VE-SDE의 이산화

6. 직접 확인해 보기 (실행 가능)

아래 셀들은 **numpy + matplotlib** 만으로 동작합니다. GPU도, 데이터 다운로드도, 학습도 필요 없습니다.
그래프 라벨은 폰트 문제를 피하기 위해 영문으로 두었습니다.

```

import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(0)
plt.rcParams["figure.dpi"] = 110
plt.rcParams["axes.grid"] = True
plt.rcParams["grid.alpha"] = 0.3

# --- DDPM 스케줄 (실습 노트북과 동일한 설정) ---
T = 300
beta_min, beta_max = 1e-4, 2e-2

betas = np.linspace(beta_min, beta_max, T)
alphas = 1.0 - betas
alpha_bars = np.cumprod(alphas)

print(f"T = {T}, beta: {betas[0]:.5f} -> {betas[-1]:.5f}")
print(f"alpha_bar_1 = {alpha_bars[0]:.6f}, alpha_bar_T = {alpha_bars[-1]:.6f}")
print(f"x_T의 신호 비중 sqrt(alpha_bar_T) = {np.sqrt(alpha_bars[-1]):.4f} (0에 가까울수록 완전 노이즈)")
  
```

실행 결과

```

T = 300, beta: 0.00010 -> 0.02000
alpha_bar_1 = 0.999900, alpha_bar_T = 0.048058
x_T의 신호 비중 sqrt(alpha_bar_T) = 0.2192 (0에 가까울수록 완전 노이즈)
  
```

6.1 노이즈 스케줄 — $\beta_t, \bar{\alpha}_t, \text{SNR}$

$\bar{\alpha}_t$ 는 "t 시점에 원본이 얼마나 살아남았는가"입니다. 신호대잡음비는

$$\text{SNR}(t) = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t}$$

로 정의되며, 로그 스케일에서 거의 단조 감소하는 직선에 가깝습니다.

```
fig, ax = plt.subplots(1, 3, figsize=(13, 3.4))
t = np.arange(1, T + 1)

ax[0].plot(t, betas, color="crimson")
ax[0].set_title(r"linear $\beta_t$ schedule"); ax[0].set_xlabel("t")

ax[1].plot(t, alpha_bars, label=r"$\bar{\alpha}_t$ (signal)")
ax[1].plot(t, 1 - alpha_bars, label=r"$1 - \bar{\alpha}_t$ (noise)")
ax[1].plot(t, np.sqrt(alpha_bars), "--", label=r"$\sqrt{\bar{\alpha}_t}$")
ax[1].set_title("signal vs noise share"); ax[1].set_xlabel("t"); ax[1].legend(fontsize=8)

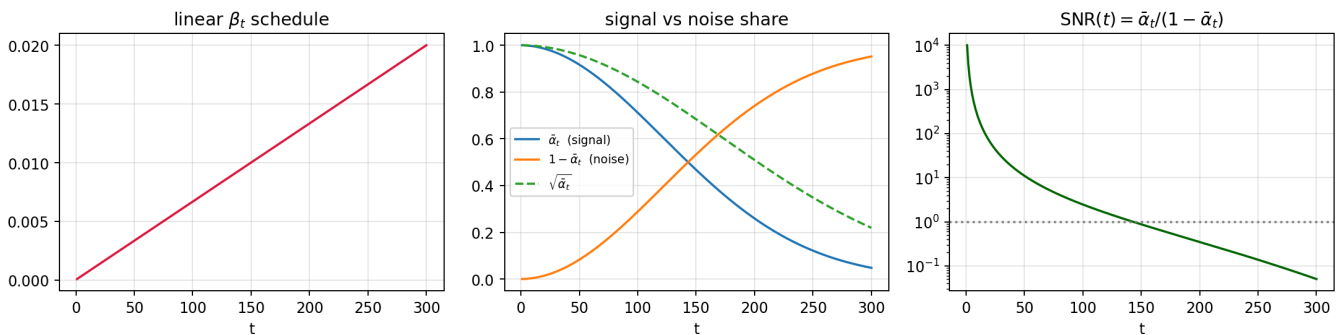
snr = alpha_bars / (1 - alpha_bars)
ax[2].semilogy(t, snr, color="darkgreen")
ax[2].axhline(1.0, ls=":", color="gray")
ax[2].set_title(r"SNR$(t)=\bar{\alpha}_t/(1-\bar{\alpha}_t)$"); ax[2].set_xlabel("t")

plt.tight_layout(); plt.show()

cross = int(np.argmin(np.abs(snr - 1.0))) + 1
print(f"SNR = 1 이 되는 지점: t = {cross} (이 이후로는 노이즈가 신호보다 우세)")
```

실행 결과

SNR = 1 이 되는 지점: t = 143 (이 이후로는 노이즈가 신호보다 우세)



6.2 순방향 확산을 2D 토이 데이터로 보기

$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$ 를 그대로 적용합니다.

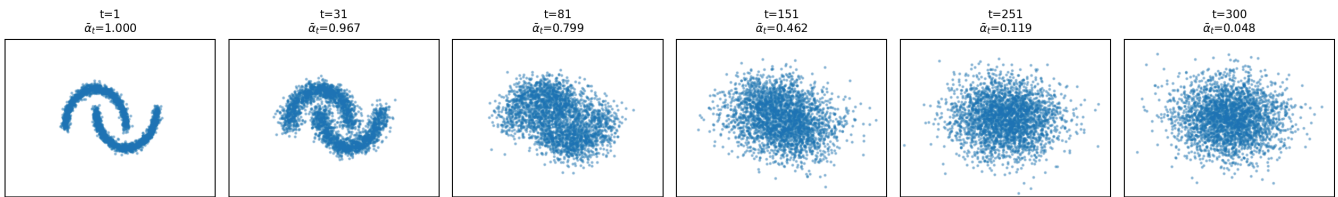
복잡한 구조(두 초승달)가 점점 표준정규분포로 뭉개지는 과정입니다.

```
def two_moons(n, noise=0.06, seed=1):
    g = np.random.default_rng(seed)
    m = n // 2
    th1 = np.pi * g.random(m)
    th2 = np.pi * g.random(n - m)
    outer = np.stack([np.cos(th1), np.sin(th1)], 1)
    inner = np.stack([1 - np.cos(th2), 0.5 - np.sin(th2)], 1)
    X = np.concatenate([outer, inner], 0)
    X = X + noise * g.standard_normal(X.shape)
    return (X - X.mean(0)) / X.std(0)      # 표준화

x0 = two_moons(3000)
show_t = [0, 30, 80, 150, 250, 299]      # 0-indexed

fig, axes = plt.subplots(1, len(show_t), figsize=(15, 2.7))
for ax, ti in zip(axes, show_t):
    ab = alphaBars[ti]
    eps = rng.standard_normal(x0.shape)
    xt = np.sqrt(ab) * x0 + np.sqrt(1 - ab) * eps
    ax.scatter(xt[:, 0], xt[:, 1], s=2, alpha=0.35)
    ax.set_title(f"t={ti+1}\n" + r"$\bar{\alpha}_t$=" + f"{ab:.3f}", fontsize=9)
    ax.set_xlim(-4, 4); ax.set_ylim(-4, 4); ax.set_xticks([]); ax.set_yticks([])
plt.suptitle("Forward diffusion: data -> N(0, I)", y=1.08)
plt.tight_layout(); plt.show()
```

Forward diffusion: data -> N(0, I)



6.3 ★ 사후분포 계수 수치 검증 (§ 3.5)

유도 결과가 맞는지 몬테카를로로 확인합니다.

$$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t, \quad \tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\bar{\alpha}_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t$$

방법: $x_{t-1} \sim q(\cdot | x_0)$, $x_t \sim q(\cdot | x_{t-1})$ 를 대량 샘플링한 뒤, x_t 가 특정 값 근처인 샘플만 골라 x_{t-1} 의 경험적 평균·분산을 봅니다.

```

t_idx = 120                # 0-indexed -> 실제 t = 121
x0_val = 1.0
xt_tgt = 0.55              # 조건으로 걸 x_t 값

a_t = alphas[t_idx]
b_t = betas[t_idx]
ab_t = alpha_bars[t_idx]
ab_tm = alpha_bars[t_idx - 1]

# 이론값
tilde_beta = (1 - ab_tm) / (1 - ab_t) * b_t
tilde_mu = (np.sqrt(ab_tm) * b_t / (1 - ab_t)) * x0_val \
           + (np.sqrt(a_t) * (1 - ab_tm) / (1 - ab_t)) * xt_tgt

# 몬테카를로
N = 4_000_000
xtm1 = np.sqrt(ab_tm) * x0_val + np.sqrt(1 - ab_tm) * rng.standard_normal(N)
xt = np.sqrt(a_t) * xtm1 + np.sqrt(b_t) * rng.standard_normal(N)

band = 0.01
mask = np.abs(xt - xt_tgt) < band
sel = xtm1[mask]

print(f"t = {t_idx+1}, x0 = {x0_val}, x_t ~={xt_tgt} (조건 만족 샘플 {mask.sum():,}개)\n")
print(f" mean 이론 {tilde_mu:.6f} 실측 {sel.mean():.6f} 차이 {abs(tilde_mu-
sel.mean()):.2e}")
print(f" var 이론 {tilde_beta:.6f} 실측 {sel.var():.6f} 차이 {abs(tilde_beta-
sel.var()):.2e}")

# A = 1/sigma^2 계산
A = a_t / b_t + 1.0 / (1 - ab_tm)
print(f"\n 1/A = {1/A:.6f} == tilde_beta = {tilde_beta:.6f} -> {np.isclose(1/A,
tilde_beta)}")

```

실행 결과

t = 121, x0 = 1.0, x_t ~= 0.55 (조건 만족 샘플 48,115개)

mean	이론 0.557011	실측 0.557004	차이 7.16e-06
var	이론 0.007984	실측 0.007907	차이 7.74e-05

1/A = 0.007984 == tilde_beta = 0.007984 -> True

6.4 스코어 필드 — "데이터가 많은 방향"

혼합 가우시안의 스코어 $\nabla_x \log p(\mathbf{x})$ 를 해석적으로 계산해 화살표로 그립니다.

밝은 곳(고밀도)을 향해 화살표가 모이고, 봉우리 정점에서는 길이가 0에 가까워집니다.

```

centers = np.array([[-1.6, -1.0], [1.7, -0.6], [0.1, 1.7]])
w       = np.array([0.4, 0.35, 0.25])
s2      = 0.35 ** 2

# 혼합 가우시안의 log p 와 score.
# extra_sigma > 0 이면 분산이 s2 + sigma^2 로 커진 평활화 분포 p_sigma 를 쓴다 (annealed 용).
def log_p_and_score(X, extra_sigma=0.0):
    v = s2 + extra_sigma ** 2
    d = X[:, None, :] - centers[None, :, :]          # (N, K, 2)
    q = -np.sum(d ** 2, -1) / (2 * v) + np.log(w)      # (N, K)
    mx = q.max(1, keepdims=True)
    e = np.exp(q - mx)
    r = e / e.sum(1, keepdims=True)                  # responsibility
    score = -np.sum(r[:, :, None] * d, 1) / v
    logp = (mx[:, 0] + np.log(e.sum(1))) - np.log(2 * np.pi * v)
    return logp, score

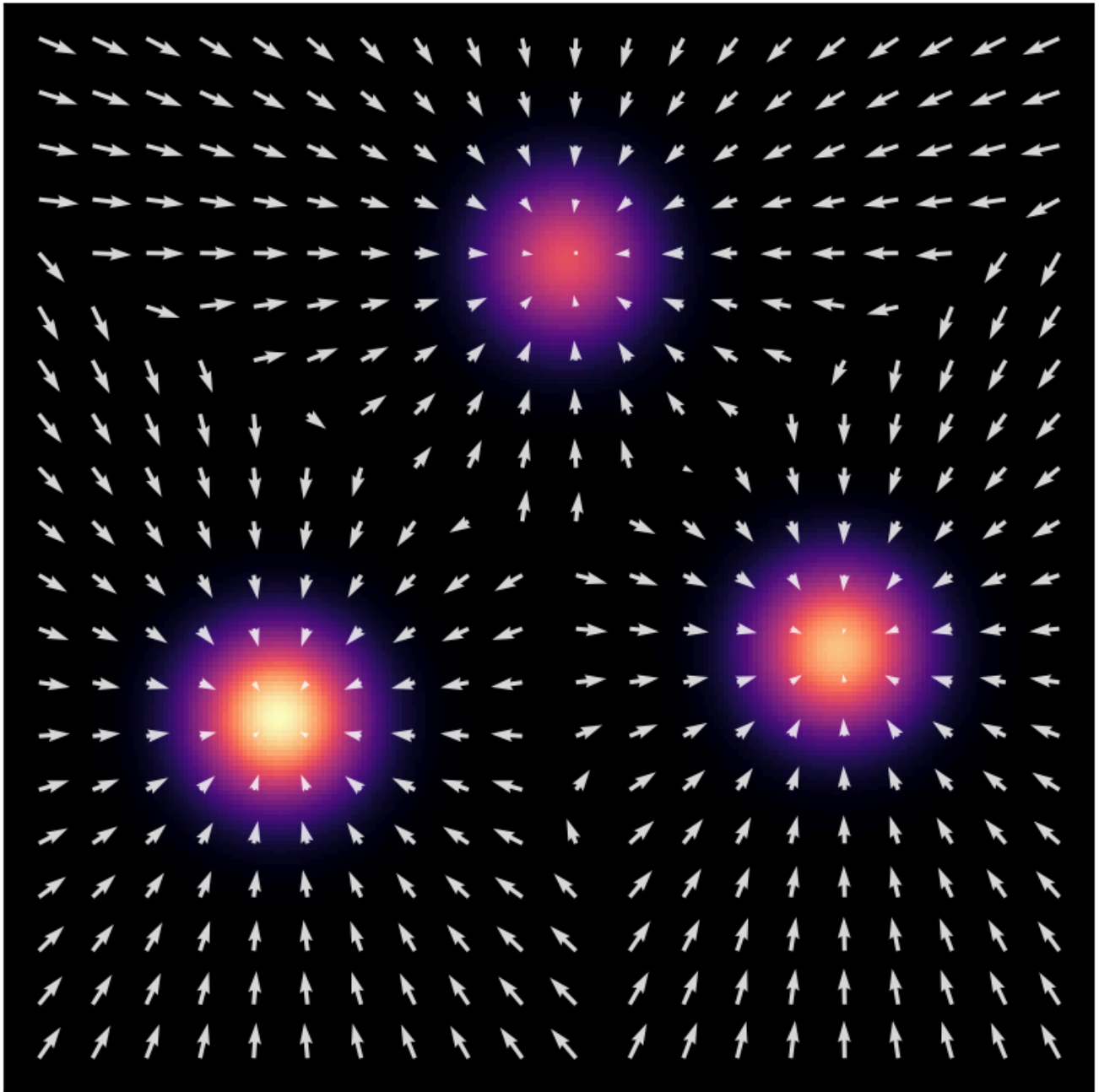
gx = np.linspace(-3.2, 3.2, 220)
GX, GY = np.meshgrid(gx, gx)
P = np.stack([GX.ravel(), GY.ravel()], 1)
logp, _ = log_p_and_score(P)

qx = np.linspace(-3.0, 3.0, 20)
QX, QY = np.meshgrid(qx, qx)
Q = np.stack([QX.ravel(), QY.ravel()], 1)
_, sc = log_p_and_score(Q)
sc_n = sc / (np.linalg.norm(sc, axis=1, keepdims=True) + 8.0) # 시각화용 정규화

plt.figure(figsize=(5.4, 5))
plt.imshow(np.exp(logp).reshape(GX.shape), extent=[-3.2, 3.2, -3.2, 3.2],
           origin="lower", cmap="magma")
plt.quiver(QX, QY, sc_n[:, 0].reshape(QX.shape), sc_n[:, 1].reshape(QX.shape),
           color="white", alpha=0.85, width=0.004)
plt.title(r"density  $p(x)$  + score field  $\nabla_x \log p(x)$ ")
plt.xticks([]); plt.yticks([]); plt.tight_layout(); plt.show()

```

density $p(x)$ + score field $\nabla_x \log p(x)$



6.5 Langevin Dynamics — 노이즈항의 역할

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \epsilon \nabla \log p(\mathbf{x}_i) + \underbrace{\sqrt{2\epsilon} \mathbf{z}_i}_{\text{이 항의 유무}}$$

노이즈를 빼면 샘플이 가장 가까운 봉우리 정점으로 전부 빨려 들어가 세 개의 점으로 뭉칩니다(분포가 아니라 최빈값 3개).

노이즈를 넣으면 각 봉우리 주변의 퍼짐(local spread)이 실제 분포와 같아집니다.

```

def langevin(x, n_steps=800, eps=0.01, noise=True, extra_sigma=0.0, seed=3):
    g = np.random.default_rng(seed)
    x = x.copy()
    for _ in range(n_steps):
        _, s = log_p_and_score(x, extra_sigma)
        x = x + eps * s
        if noise:
            x = x + np.sqrt(2 * eps) * g.standard_normal(x.shape)
    return x

x_init = rng.standard_normal((2500, 2)) * 2.0
x_det = langevin(x_init, noise=False)
x_lan = langevin(x_init, noise=True)

fig, ax = plt.subplots(1, 3, figsize=(12, 3.8))
for a, X, ttl in zip(ax,
                    [x_init, x_det, x_lan],
                    ["init: N(0, 2I)",
                    "gradient only -> collapses to 3 points",
                    "Langevin -> correct local spread"]):
    a.scatter(X[:, 0], X[:, 1], s=3, alpha=0.3)
    a.scatter(centers[:, 0], centers[:, 1], marker="x", c="red", s=70)
    a.set_xlim(-3.2, 3.2); a.set_ylim(-3.2, 3.2)
    a.set_title(ttl, fontsize=9); a.set_xticks([]); a.set_yticks([])
plt.tight_layout(); plt.show()

def spread(X):
    # 가장 가까운 중심까지의 평균 거리
    return np.linalg.norm(X[:, None, :] - centers[None, :], axis=2).min(1).mean()

print(f"true (sigma=0.35) 기준 평균 거리 : ~0.44")
print(f"gradient only : {spread(x_det):.4f} <- 0에 수렴 = 붕괴")
print(f"Langevin MCMC : {spread(x_lan):.4f} <- 실제 퍼짐 재현")

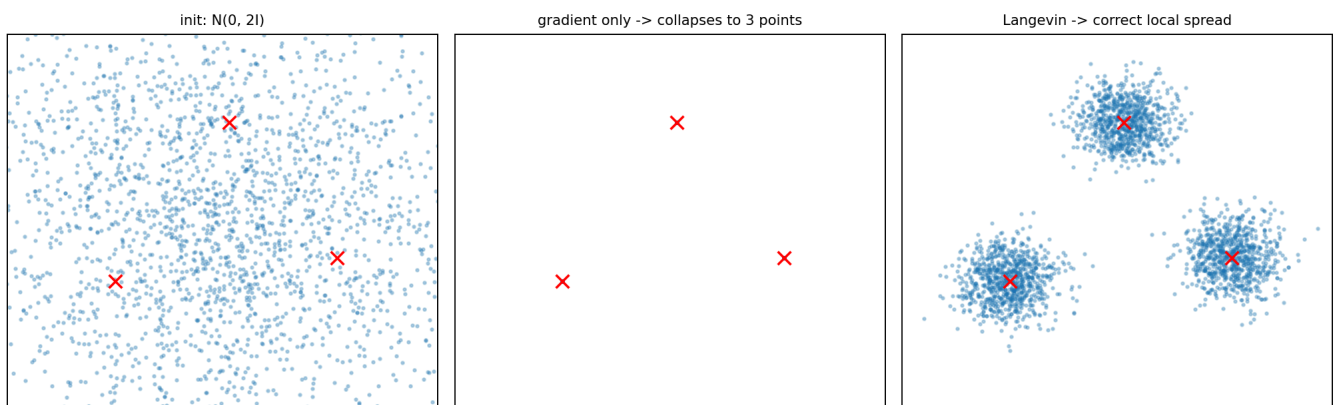
```

실행 결과

```

true (sigma=0.35) 기준 평균 거리 : ~0.44
gradient only : 0.0000 <- 0에 수렴 = 붕괴
Langevin MCMC : 0.4497 <- 실제 퍼짐 재현

```



6.6 ★ 왜 Annealed Langevin이 필요한가 (§ 4.6의 실증)

위 Langevin은 국소 형태는 맞지만 모드 간 비율(0.40 / 0.35 / 0.25)은 재현하지 못합니다. 봉우리 사이의 저밀도 골짜기가 장벽이 되어 샘플이 처음 도착한 봉우리에 갇히기 때문입니다.

해결책이 **Annealed Langevin**입니다. 짙은 안개(σ 큼)에서 시작하면 밀도가 뭉개져 장벽이 사라지므로 샘플이 자유롭게 이동하고, 안개를 서서히 걷으면($\sigma \downarrow$) 제자리를 찾아갑니다. 스텝 크기도 $\alpha_i = \epsilon \cdot \sigma_i^2 / \sigma_L^2$ 로 함께 줄입니다 (NCSN 방식).

$$p_\sigma(\mathbf{x}) = \int p(\mathbf{y}) \mathcal{N}(\mathbf{x}; \mathbf{y}, \sigma^2 \mathbf{I}) d\mathbf{y}$$

혼합 가우시안에서는 각 성분의 분산이 $s^2 + \sigma^2$ 로 커진 형태이므로 해석적으로 계산됩니다.

```
def annealed_langevin(x, sigmas=np.geomspace(2.0, 0.02, 12),
                    n_per_level=200, eps0=2e-4, seed=5):
    g = np.random.default_rng(seed)
    x = x.copy()
    sigma_L = sigmas[-1]
    for s in sigmas:
        alpha = eps0 * (s ** 2) / (sigma_L ** 2)          # 노이즈 레벨별 스텝 크기
        for _ in range(n_per_level):
            _, sc = log_p_and_score(x, s)
            x = x + alpha * sc + np.sqrt(2 * alpha) * g.standard_normal(x.shape)
    return x

x_ann = annealed_langevin(x_init)

def mode_share(X):
    d = np.linalg.norm(X[:, None, :] - centers[None, :], axis=2)
    return np.bincount(d.argmax(1), minlength=3) / len(X)

fig, ax = plt.subplots(1, 2, figsize=(8.4, 3.9))
for a, X, ttl in zip(ax, [x_lan, x_ann],
                    ["plain Langevin", "annealed Langevin"]):
    a.scatter(X[:, 0], X[:, 1], s=3, alpha=0.3)
    a.scatter(centers[:, 0], centers[:, 1], marker="x", c="red", s=70)
    a.set_xlim(-3.2, 3.2); a.set_ylim(-3.2, 3.2)
    a.set_title(ttl, fontsize=10); a.set_xticks([]); a.set_yticks([])
plt.tight_layout(); plt.show()

print("true weights      :", np.round(w, 3))
print("plain Langevin    :", np.round(mode_share(x_lan), 3), " <- 초기 위치에 갇힘")
print("annealed Langevin :", np.round(mode_share(x_ann), 3), " <- 비율 재현")
```

실행 결과

```
true weights      : [0.4  0.35 0.25]
plain Langevin    : [0.337 0.326 0.337] <- 초기 위치에 갇힘
annealed Langevin : [0.403 0.352 0.245] <- 비율 재현
```



이것이 Day 3 전체의 동기입니다. 노이즈 레벨을 몇 단계로 나눌지가 하이퍼파라미터로 남고, 레벨 사이가 이산적으로 끊겨 있다는 한계 → "무한히 연속으로 만들자" → SDE.

6.7 VP-SDE vs VE-SDE — 분산 거동 비교

- **VP**: $\text{Var}(x_t) = \bar{\alpha}_t \text{Var}(x_0) + (1 - \bar{\alpha}_t) \rightarrow 1$ 로 유지
- **VE**: $\text{Var}(x_t) = \text{Var}(x_0) + \sigma_t^2, \sigma_t^2 = \frac{\sigma^{2t} - 1}{2 \log \sigma} \rightarrow$ 폭발 (실습의 $\sigma = 25$)

```
tt = np.linspace(1e-3, 1.0, 400)

# VP (연속시간 근사: beta(t) = b_min + t*(b_max-b_min))
b_min, b_max = 0.1, 20.0
log_ab_vp = -(b_min * tt + 0.5 * (b_max - b_min) * tt ** 2)
var_vp = np.exp(log_ab_vp) * 1.0 + (1 - np.exp(log_ab_vp))

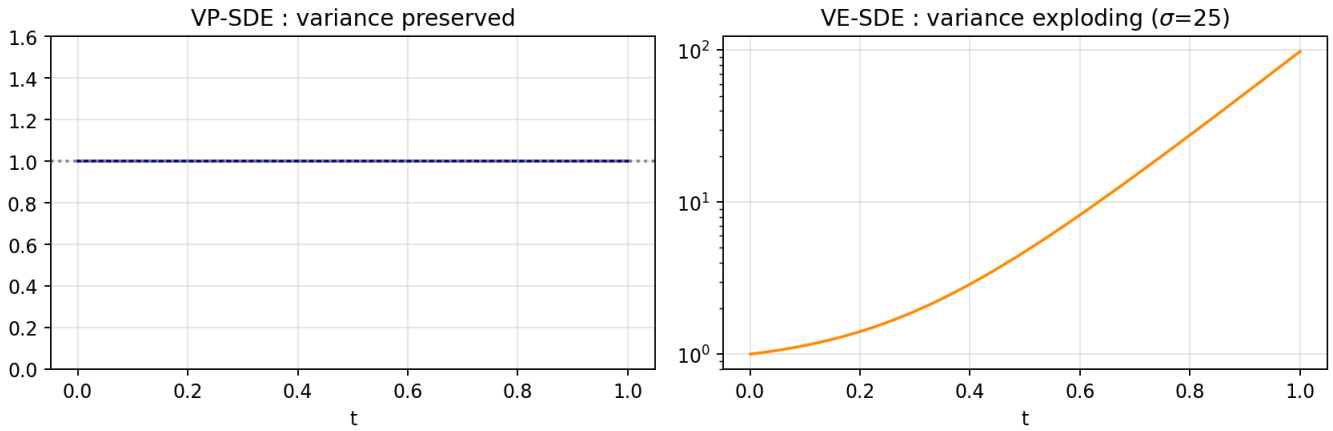
# VE
sigma_ve = 25.0
var_ve = 1.0 + (sigma_ve ** (2 * tt) - 1.0) / (2 * np.log(sigma_ve))

fig, ax = plt.subplots(1, 2, figsize=(10, 3.4))
ax[0].plot(tt, var_vp, color="navy"); ax[0].axhline(1, ls=":", c="gray")
ax[0].set_title("VP-SDE : variance preserved"); ax[0].set_xlabel("t"); ax[0].set_ylim(0, 1.6)
ax[1].semilogy(tt, var_ve, color="darkorange")
ax[1].set_title(r"VE-SDE : variance exploding (\sigma=25)"); ax[1].set_xlabel("t")
plt.tight_layout(); plt.show()

print(f"VP Var(x_0)=1.00 -> Var(x_1)={var_vp[-1]:.3f}")
print(f"VE Var(x_0)=1.00 -> Var(x_1)={var_ve[-1]:.1f}")
```

실행 결과

VP Var(x₀)=1.00 -> Var(x₁)=1.000
VE Var(x₀)=1.00 -> Var(x₁)=97.9



7. 자주 헷갈리는 지점 Q&A

Q1. α_t 와 $\bar{\alpha}_t$ 를 자주 헷갈립니다.

$\alpha_t = 1 - \beta_t$ 는 한 스텝의 생존율, $\bar{\alpha}_t = \prod_{s \leq t} \alpha_s$ 는 0부터 t까지 누적 생존율입니다. 한 스텝 전이식에는 α_t 가, $\mathbf{x}_0 \rightarrow \mathbf{x}_t$ 직행식에는 $\bar{\alpha}_t$ 가 들어갑니다.

Q2. DDPM의 U-Net은 왜 t 를 입력으로 받나요?

같은 노이즈 이미지라도 $t = 10$ 일 때와 $t = 900$ 일 때 제거해야 할 노이즈의 양이 완전히 다릅니다. 하나의 네트워크가 T 개의 서로 다른 디노이저 역할을 겸하려면 "지금 몇 번째 단계인가"를 알아야 합니다.

Q3. 역방향인데 왜 노이즈를 또 더하나요? ($+\sigma_t \mathbf{z}$)

매번 평균값만 취하면 모든 샘플이 같은 결과(분포의 평균)로 수렴합니다. 우리가 원하는 것은 분포에서 샘플링하는 것이지 평균을 구하는 게 아닙니다. DDIM에서 $\sigma_t = 0$ 으로 두면 다양성이 초기 노이즈 $\mathbf{x}_T$ 에서만 나옵니다.

Q4. Score와 노이즈 예측의 관계를 한 줄로?

$s_\theta(\mathbf{x}_t, t) = -\frac{\epsilon_\theta(\mathbf{x}_t, t)}{\sqrt{1 - \bar{\alpha}_t}}$. 같은 것을 부호와 스케일만 달리해 부른 이름입니다.

Q5. DDIM이 DDPM보다 빠르는데 왜 재학습이 필요 없나요?

DDIM은 학습 목표(L_{simple})와 네트워크를 그대로 두고 샘플링 수식만 바꿉니다. 학습은 "임의의 t 에서 노이즈 맞추기"이므로 어떤 순서·간격으로 샘플링하든 무관합니다.

Q6. VAE의 KL과 DDPM의 L_T 는 같은 것인가요?

역할이 같습니다. 둘 다 "최종 잠재 상태가 사전분포를 따르게 하라"는 항입니다. 차이는 DDPM의 L_T 에는 학습 파라미터가 없어 상수로 무시된다는 점입니다 (순방향이 고정되어 있으므로).

Q7. PF-ODE가 결정론적이면 다양성은 어디서 오나요?

초기 노이즈 $\mathbf{x}_T$ 하나에서만 옵니다. 그래서 $\mathbf{x}_T \leftrightarrow \mathbf{x}_0$ 가 1:1 대응이 되고, 이것이 정확한 역전(inversion)과 편집을 가능하게 합니다.

Q8. VE-SDE의 분산이 폭발해도 괜찮은가요?

괜찮습니다. 목표는 $\mathbf{x}_T$ 를 표준정규로 만드는 게 아니라 어떤 알려진 분포로 만드는 것입니다. VE에서는 $\mathcal{N}(\mathbf{0}, \sigma_{\max}^2 \mathbf{I})$ 에서 샘플링을 시작합니다. 대신 저밀도 영역까지 스코어를 학습한다는 이점이 있습니다.

Q9. L_{simple} 은 가중치를 버렸는데, 그래도 되나요?

엄밀히는 VLB와 다른 목적함수입니다. 하지만 실험적으로 샘플 품질이 더 좋았고(작은 t 의 과도한 가중을 완화), 이후 연

구에서 이 가중치가 **perceptual quality에 유리한 재가중임**이 밝혀졌습니다. 우도 자체를 최적화하려면 L_{vib} 를 써야 합니다.

Q10. 어떤 순서로 복습하면 좋을까요?

① § 5.3의 세 등식 → ② § 3.2의 닫힌 형태와 § 3.5의 유도 → ③ § 4.8의 퍼터베이션 커널. 이 셋이 연결되면 나머지는 파생입니다.

8. 원본 자료 정오표 / 읽을 때 주의할 점

노트북을 그대로 따라가다 막히기 쉬운 지점만 모았습니다. 내용의 본질에는 영향이 없습니다.

위치	원문	바로잡기
<code>diffusion_분포_유도.pdf</code> p.1	$\mathcal{N}(\sqrt{1-\beta_t}, x_{t-1}, \beta_t I)$	첨표는 곱셈 기호여야 함 → $\mathcal{N}(\sqrt{1-\beta_t} x_{t-1}, \beta_t I)$
PDF p.3 (5-a)	$\exp(-\frac{1}{2B_t} \ x_t - \sqrt{\alpha_t} x\ ^2)$	$B_t = \beta_t, x = x_{t-1}$
PDF p.3 (5-b, 5-c)	$\ x - \sqrt{\alpha} x_0\ ^2$	앞의 x 는 각각 x_{t-1}, x_t
DAY2 이론 [20]	" $\alpha_t = 1 - \beta_t, \alpha_t = \prod_{s=1}^t \alpha_s$ "	두 번째는 $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$
DAY2 이론 [35]	$\hat{x}_0 = \frac{x_t - \sigma \epsilon \theta}{\alpha}$	마크다운 렌더 깨짐. $\hat{x}_0 = \frac{x_t - \sigma \epsilon \theta}{\alpha}$
DAY1 실습 [16] 디코더 설명	인코더 설명이 그대로 복사됨 (<code>FC_mean</code> , <code>FC_var</code> , mean/log_var 언급)	디코더에는 <code>FC_hidden</code> , <code>FC_hidden2</code> , <code>FC_output</code> 만 있고 <code>sigmoid</code> 로 끝남
DAY1 실습 [19]	<code>reparameterization(self, mean, var)</code> 에 <code>torch.exp(0.5*log_var)</code> 전달	인자로 들어오는 값은 분산이 아니라 표준편차 . 변수명만 오해 소지 (<code>z = mean + std*eps</code> 로 계산은 정확)
DAY1 실습 [24]	<code>total_loss / (batch_idx*x.size(0))</code>	마지막 배치가 작으면 분모가 부정확. <code>len(train_loader.dataset)</code> 이 안전
DAY2 실습 [6]	<code>img_size=(14,14,1)</code> , <code>n_timesteps=300</code>	수업용 축소판. 결과가 흐린 것은 정상
DAY3 실습	<code>flax.jax_utils.replicate</code> / <code>pmap</code>	GPU 1개 환경에서도 동작하지만, <code>batch_size % device_count == 0</code> 조건 필요

9. 용어 사전

용어	정의	처음 등장
Intractable	정규화(전체 합·적분) 계산량이 감당 불가능한 상태	Day1 § 2.2
Posterior	$p(\mathbf{z} \mathbf{x})$. "이 데이터는 잠재공간 어디쯤인가"	Day1
ELBO	로그우도의 계산 가능한 하한	Day1 § 2.3
Reparameterization	무작위성을 모델 밖으로 분리해 미분 가능하게 만드는 기법	Day1 § 2.4
Posterior collapse	KL 항이 과해져 $q_\phi \rightarrow p(\mathbf{z})$ 가 되고 $\mathbf{z}$ 가 정보를 잃는 현상	Day1
닫힌 형태 (Closed form)	적분·반복 없이 대입만으로 답이 나오는 명시적 수식	Day2 § 3.2
마르코프 성질	다음 상태가 직전 상태에만 의존	Day2 § 3.5
VLB / L_{simple}	변분하한 / 이를 단순화한 노이즈 MSE	Day2 § 3.4
Score	$\nabla_{\mathbf{x}} \log p(\mathbf{x})$. 밀도가 커지는 방향	Day3 § 4.2
Fisher Divergence	참 스코어와 예측 스코어의 기대 제곱오차	Day3 § 4.3
DSM	Denoising Score Matching. 직접 섞은 노이즈를 정답으로 삼아 스코어 학습	Day3 § 4.4
Langevin Dynamics	스코어 + 노이즈로 분포에서 샘플링하는 MCMC	Day3 § 4.5
Perturbation Kernel	$p_{0t}(\mathbf{x}_t \mathbf{x}_0)$. t 시점 노이즈 상태의 분포	Day3 § 4.8
Drift / Diffusion 계수	SDE의 결정론적 흐름 / 무작위 강도	Day3 § 4.8
Wiener process	브라운 운동. 평균 0, 분산 dt	Day3 § 4.8
VP / VE	Variance Preserving(=DDPM) / Variance Exploding(=NCSN)	Day3 § 4.9
Fokker-Planck	입자들의 무작위 운동에서 집단 밀도 의 시간 변화를 기술하는 식	Day3 § 4.11
PF-ODE	무작위성을 제거하고도 밀도 변화가 동일한 결정론적 ODE	Day3 § 4.11
SNR (샘플러)	PC 샘플러 교정 강도 하이퍼파라미터 (0.1~0.2)	Day3 § 4.13

10. 다음 단계 — Flow Matching

3일 과정의 종착점이자 2026년 현재의 표준입니다.

기존 Diffusion/Score 모델이 **안갯속에서 굽이굽이 산맥을 내려오는 방식**이었다면, Flow Matching은 이렇게 묻습니다.

"왜 꼭 돌아가야 하지? 노이즈와 데이터 사이에 **직선 고속도로**를 뚫으면 안 되나?"

- 노이즈 $\mathbf{x}_1$ 과 데이터 $\mathbf{x}_0$ 를 잇는 **가장 효율적인 직선 경로**를 상정
- 모델은 그 경로 위에서의 **속도장(velocity field)** 을 직접 학습
- 경로가 직선에 가까우므로 **10~20 스텝**만에 복원 완료

- **Stable Diffusion 3(SD3)** 가 이 기법을 핵심으로 채택

	Diffusion / Score	Flow Matching
학습 대상	$\epsilon_\theta / s_\theta$	속도장 $v_\theta(\mathbf{x}, t)$
경로	곡선 (확률적 또는 결정론적)	거의 직선
스텝 수	50~1000	10~20

11. 참고 자료

논문

- Kingma & Welling, *Auto-Encoding Variational Bayes* (2013)
- Ho, Jain & Abbeel, *Denoising Diffusion Probabilistic Models* (2020)
- Song & Ermon, *Generative Modeling by Estimating Gradients of the Data Distribution* (NCSN, 2019)
- Song et al., *Score-Based Generative Modeling through Stochastic Differential Equations* — [arXiv:2011.13456](https://arxiv.org/abs/2011.13456) (PF-ODE 유도는 Appendix D)
- Song, Meng & Ermon, *Denoising Diffusion Implicit Models* (DDIM, 2021)
- Rombach et al., *High-Resolution Image Synthesis with Latent Diffusion Models* (LDM, 2022)
- Lipman et al., *Flow Matching Guide and Code* — [arXiv:2412.06264](https://arxiv.org/abs/2412.06264)

해설

- Yang Song, *Generative Modeling by Estimating Gradients of the Data Distribution* — <https://yang-song.net/blog/2021/score/>
- Lilian Weng, *What are Diffusion Models?* — <https://lilianweng.github.io/posts/2021-07-11-diffusion-models/>
- The AI Summer, *How diffusion models work* — <https://theaisummer.com/diffusion-models/>
- DDPM 수식 유도 영상 — <https://www.youtube.com/watch?v=ybvJbvllgJk>
- Flow Matching 저자 강의 (NeurIPS 2024) — <https://neurips.cc/virtual/2024/tutorial/99531>

원 강의자료: 박광석 (모두의연구소) · 본 통합노트는 DAY1~3 이론·실습 노트북과 분포 유도 PDF를 하나의 흐름으로 재구성한 것입니다.